PROJECT REPORT

INTELLIGENT PARKING ASSISTANCE SYSTEM

Using STM32F103RB (ARM Cortex-M3)

Domain: Embedded Systems and IoT

Tools: STM32CubeMX, STM32CubeIDE, STM32 HAL, Embedded C

TEAM MEMBERS

Roll Number	Name
2510080017	Shirisha
2510080023	Keerthi Varna
2510080036	Shravika
2510080062	ArunReddy
2510080051	A. Pranith Reddy

Microcontroller: STM32F103RB (ARM Cortex-M3)

Prepared by adapting the supplied Traffic Signal Controller report format to the parking-assistance problem statement.

Table of Contents

1. 1. Abstract
2. 2. Introduction
3. 3. Problem Statement
4. 4. Objectives
5. 5. Hardware Requirements
6. 6. Software Requirements
7. 7. System Architecture
8. 8. Proposed Pin Configuration
9. 9. Working Principle
10. 10. Interrupt Handling and NVIC
11. 11. Cortex-M3 Memory, Peripheral Registers and Addresses
12. 12. STM32CubeMX Configuration Procedure
13. 13. STM32CubeIDE Implementation Plan
14. 14. Proposed Algorithm
15. 15. Testing and Validation Plan
16. 16. Expected Results
17. 17. Advantages
18. 18. Limitations and Safety Notes
19. 19. Future Enhancements
20. 20. Conclusion
21. 21. Final Project Summary
22. References and Appendices

1. Abstract

The Intelligent Parking Assistance System is an embedded-system prototype that helps a driver park a vehicle safely. An ultrasonic sensor measures the distance between the vehicle and an obstacle, while an IR sensor detects vehicle presence. The STM32F103RB microcontroller processes these inputs and controls red, yellow and green LEDs to show the parking condition. A timer measures the ultrasonic echo pulse, another timer generates PWM for a DC motor that simulates a parking barrier, and a relay represents the parking gate. A push button is handled through an external interrupt. NVIC priorities organize interrupt handling, and UART sends distance and parking-status messages to a serial monitor. The project is configured with STM32CubeMX and developed in STM32CubeIDE using Embedded C and the STM32 HAL library.

2. Introduction

Parking in a tight space can be difficult because the driver must estimate the distance from nearby objects. A simple electronic assistant can measure the distance and provide visual feedback before the vehicle reaches an obstacle.

This project demonstrates a small parking assistant using an STM32F103RB microcontroller. It combines ultrasonic distance measurement, IR vehicle detection, LED indicators, timer-generated PWM, a relay output, a push-button interrupt and UART monitoring. It is a learning prototype and is not intended to replace a certified vehicle safety system.

3. Problem Statement

Develop a mini Intelligent Parking Assistance System using ultrasonic and IR sensors. The system must measure obstacle distance from the ultrasonic echo pulse width, detect vehicle presence using the IR sensor, and control red/yellow/green LEDs according to the measured distance. It must generate PWM using a timer to drive a DC motor as a parking-barrier simulation, control a relay to represent the parking gate, handle a push button through an external interrupt, assign suitable NVIC priorities to sensor and user interrupts, and transmit distance and parking status through UART. The report must also explain how Cortex-M3 memory maps, peripheral registers, GPIO and timer addresses relate to the implementation.

4. Objectives

1. Configure the ultrasonic sensor trigger pin as a GPIO output and measure echo time using a hardware timer.
2. Calculate obstacle distance from the measured echo pulse width.
3. Detect vehicle presence using an IR sensor.
4. Show parking guidance using red, yellow and green LEDs.
5. Generate timer-based PWM to operate a DC motor through a suitable motor driver.
6. Control a relay module to represent the parking gate.
7. Configure a push button as an EXTI interrupt and set NVIC priorities.
8. Send measured distance and parking status through UART.
9. Understand the STM32F103RB memory map and peripheral-register access.

5. Hardware Requirements

STM32F103RB development board — main controller.

HC-SR04 ultrasonic sensor — measures distance to an obstacle.

IR obstacle/vehicle sensor module — detects vehicle presence.

Red, yellow and green LEDs with individual 220–330 Ω current-limiting resistors.

Push button — user input through EXTI.

DC motor and a compatible motor-driver module, such as an L298N module — barrier simulation.

Relay module with driver — gate representation; use a module suitable for the supply voltage.

External motor supply, breadboard, jumper wires and common ground.

USB/ST-LINK programmer and USB-UART adapter or a serial terminal for UART monitoring.

6. Software Requirements

STM32CubeMX configures the clock, GPIO, timers, EXTI, NVIC and USART1.

STM32CubeIDE is used to write, build, flash and debug the Embedded C project.

STM32 HAL provides functions for GPIO, timer input capture, PWM, UART and interrupt callbacks.

A serial terminal (for example, a UART terminal in the IDE or a PC serial application) displays distance and status messages.

7. System Architecture

The ultrasonic sensor supplies an echo pulse whose width represents the round-trip travel time of sound. The IR sensor provides a digital vehicle-present signal. The STM32 reads these inputs, calculates distance, selects a parking status and updates the LEDs. TIM3 generates PWM for the motor driver. A GPIO output controls the relay module. The push button raises an EXTI event, and USART1 reports distance and status.

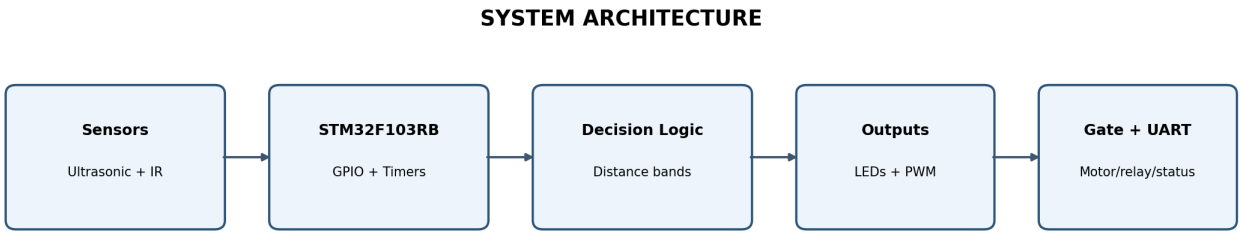


Figure 1: System architecture of the parking assistant.

8. Proposed Pin Configuration

These are proposed example pin assignments for an STM32F103RB design. Confirm pin availability against the exact development-board schematic and CubeMX pinout before wiring.

STM32 Pin	Function	Mode	Connection / Notes
PA0	Red LED	GPIO output	LED anode through 220–330 Ω resistor; cathode to GND
PA1	Yellow LED	GPIO output	LED anode through 220–330 Ω resistor; cathode to GND
PA2	Green LED	GPIO output	LED anode through 220–330 Ω resistor; cathode to GND
PB0	HC-SR04 TRIG	GPIO output	Connect to TRIG
PA8	HC-SR04 ECHO	TIM1_CH1 input capture	ECHO through divider/level shifter to 3.3 V logic
PB1	IR sensor OUT	GPIO input	Confirm output voltage and active polarity
PA6	Motor driver PWM	TIM3_CH1 PWM	Connect to driver EN/PWM input, not motor directly
PB12	Relay module IN	GPIO output	Use compatible relay module/driver
PA3	Push button	GPIO_EXTI3	Button to GND with internal/external pull-up
PA9	USART1_TX	UART transmit	Connect to USB-UART RX
PA10	USART1_RX	UART receive	Connect to USB-UART TX if receiving is needed
3V3/GND	Sensor logic supply/reference	Power/reference	Use only supply voltages supported by each module; common GND
External supply	DC motor power	Motor driver supply	Match motor/driver ratings; share GND with MCU

9. Working Principle

Step 1 — Trigger: The microcontroller sends a short HIGH pulse (about 10 µs) to the HC-SR04 TRIG pin.

Step 2 — Echo timing: The sensor raises ECHO for a duration related to the sound's round-trip travel time. A timer configured at 1 MHz counts microseconds; input capture or carefully timed edge detection records the pulse width.

Step 3 — Distance: Distance in centimetres is approximately echo_pulse_width_us / 58. This formula assumes the usual speed of sound and a round trip. A timeout must be used if no echo arrives.

Step 4 — Vehicle detection: The IR sensor output is read as present/not present. Its active-HIGH or active-LOW behaviour must be checked on the actual module.

Step 5 — LED guidance: Example thresholds are: green for distance greater than 100 cm, yellow for 30–100 cm, and red for less than 30 cm. These are adjustable demonstration thresholds, not safety-certified values. If no vehicle is detected, the system can show an idle state.

Step 6 — Barrier simulation: TIM3 generates PWM on PA6. The PWM signal drives a motor-driver module, which supplies the motor from an external power source. The relay output can enable/represent the gate. A GPIO pin must never drive the motor or relay coil directly.

Step 7 — Button: A push-button press triggers EXTI3. The interrupt handler records a request flag; the main loop processes it so the interrupt stays short.

Step 8 — UART: USART1 sends a readable line such as 'Distance: 42 cm | Vehicle: YES | Status: CAUTION'.

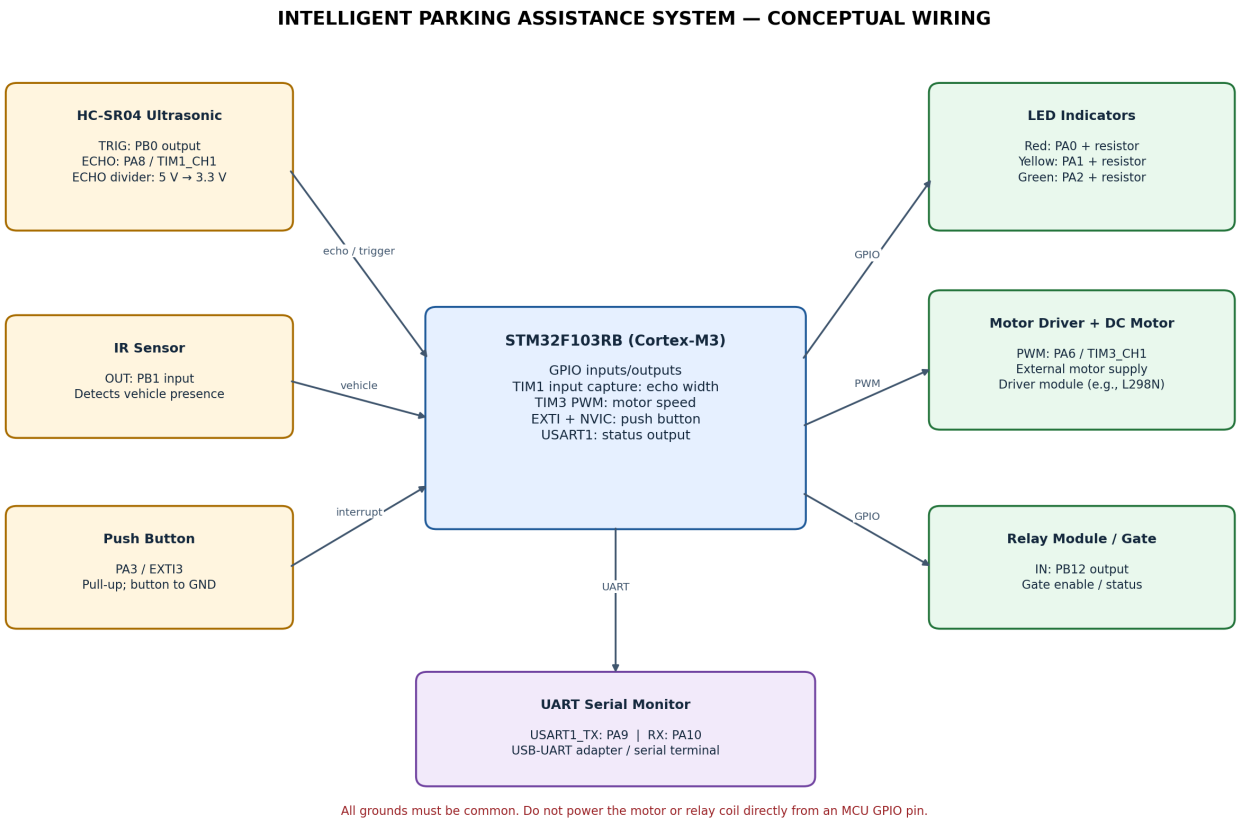


Figure 2: Conceptual circuit/wiring diagram. Follow the pin table and safety notes before wiring.

10. Interrupt Handling and NVIC

EXTI (External Interrupt/Event Controller) detects the push-button edge. Timer input-capture interrupts can record ultrasonic echo edges, while timer update events can support timeouts or periodic tasks. NVIC controls interrupt enablement and priority.

Suggested priority plan (lower numerical value means higher priority): push-button EXTI3 priority 0; ultrasonic timer capture priority 1; periodic timer/PWM support priority 2. Use the priority grouping configured in CubeMX and keep interrupt handlers short. UART printing and longer calculations should generally occur in the main loop, not inside an interrupt handler.

Important: The relay is an output and does not itself create an interrupt. If a gate-limit switch or emergency input is required, add it as a separate input and assign its EXTI line and priority.

11. Cortex-M3 Memory, Peripheral Registers and Addresses

The Cortex-M3 uses a memory-mapped I/O design. This means peripherals are controlled by reading and writing special addresses in the processor's address space, rather than using separate input/output instructions.

• Flash memory stores the program; SRAM stores variables, stack and runtime data.

• The peripheral region contains register blocks for GPIO ports, timers, USART, EXTI and other hardware.

• Each peripheral has a base address. Registers are located at offsets from that base address. For example, a GPIO port has configuration, input-data and output-data registers; a timer has control, prescaler, auto-reload, counter and capture/compare registers.

• HAL calls such as HAL_GPIO_WritePin() and HAL_TIM_PWM_Start() operate on these registers through the STM32 library. Direct register access is possible but requires the exact STM32F1 reference manual and correct bit fields.

• GPIO addresses configure/read pins; timer addresses configure the counter, prescaler, PWM and input capture; USART addresses configure baud rate and transmit/receive data. The Cortex-M3 core accesses these mapped registers over the processor bus.

Use the official STM32F103 reference manual and the selected board's datasheet for exact addresses. This report explains the relationship conceptually and does not assume one fixed address for every STM32 family.

Conceptual address map

Area	What it stores/controls	Example use
Flash	Program instructions and constants	Compiled firmware
SRAM	Variables, buffers and stack	Distance value, flags, UART text
GPIO registers	Pin mode, input and output state	Read IR; switch LEDs
Timer registers	Counter, prescaler, capture/compare, PWM	Measure ECHO; generate motor PWM
USART registers	Serial configuration and data	Transmit distance/status
EXTI/NVIC registers	External event routing and interrupt control	Button event and priorities

12. STM32CubeMX Configuration Procedure

1. Create a project for the exact STM32F103RB device/board.
2. Configure PA0, PA1 and PA2 as GPIO outputs for red, yellow and green LEDs.
3. Configure PB0 as GPIO output for HC-SR04 TRIG.
4. Configure PA8 as TIM1_CH1 input capture for HC-SR04 ECHO, with an appropriate timer clock/prescaler.
5. Configure PB1 as GPIO input for the IR sensor.
6. Configure PA6 as TIM3_CH1 PWM output for the motor driver.
7. Configure PB12 as GPIO output for the relay module.
8. Configure PA3 as GPIO_EXTI3 for the push button and select the correct edge/pull-up or pull-down.
9. Configure USART1 asynchronous mode on PA9 (TX) and PA10 (RX), with a suitable baud rate such as 115200 bit/s.
10. Enable the required timer and EXTI interrupts in NVIC and set the agreed priorities.
11. Verify the system clock and timer clock calculations; generate the code for STM32CubeIDE.

13. STM32CubeIDE Implementation Plan

1. Open the generated project and check GPIO, TIM1, TIM3 and USART1 initialization.
2. Start TIM1 input capture/echo measurement and start TIM3 PWM.
3. Generate a 10 µs trigger pulse and measure the ECHO pulse width with a timeout.
4. Calculate distance and validate that the measurement is within the sensor's supported range.
5. Read the IR input and determine whether a vehicle is present.
6. Select the LED state from the configured distance thresholds.
7. Update the PWM duty cycle as required for the barrier demonstration and control the relay output.
8. In the EXTI callback, set a button-request flag only; handle the action in the main loop.
9. Send distance, vehicle detection and status messages using HAL_UART_Transmit() or a non-blocking UART method.
10. Build, flash and test each peripheral separately before testing the integrated system.

14. Proposed Algorithm

START → Initialize HAL, clock, GPIO, TIM1, TIM3, EXTI and USART1 → Start timer capture and PWM → Send ultrasonic trigger pulse → Measure echo pulse width or detect timeout → Calculate distance → Read IR sensor → Decide parking status → Update red/yellow/green LEDs → Control motor-driver PWM and relay output as needed → Send distance/status through UART → Check button request → Repeat.

Flow: START → Initialize peripherals → Trigger ultrasonic sensor → Measure echo/timeout → Calculate distance and read IR → Choose status and update LEDs → Control PWM motor and relay → Send UART status → Repeat.

15. Testing and Validation Plan

1. Power-on test: the microcontroller starts and outputs are set to a safe initial state.

2. Ultrasonic test: compare calculated distance with a ruler at several distances within the sensor's range.

3. No-echo test: verify that a timeout is reported instead of using an invalid distance.

4. IR test: move an object into and out of the sensor range and confirm vehicle detection.

5. LED test: verify green, yellow and red transitions at the selected thresholds.

6. PWM test: check the PWM waveform and confirm the motor runs only through the motor driver.

7. Relay test: confirm the relay module switches as intended and is driven using the correct voltage/current.

8. Button test: verify EXTI3 detects a press and that the NVIC priority configuration is active.

9. UART test: verify distance, vehicle presence and parking status appear correctly in the serial terminal.

10. Integration test: operate all components repeatedly and check for missed echoes, unstable sensor readings or unexpected output changes.

These are planned validation steps; results should be marked as completed only after the physical circuit has been tested.

Test	Expected observation
Ultrasonic at known distances	Calculated distance close to ruler measurement
No echo	Timeout handled safely
IR blocked/unblocked	Vehicle state changes
Distance threshold crossed	Correct LED turns on
PWM output	PWM visible; motor uses driver
Button press	EXTI request recorded
UART terminal	Distance and status readable
Repeated integration test	No unexpected output changes

16. Expected Results

The system is expected to measure obstacle distance using the ultrasonic echo pulse, detect vehicle presence using the IR sensor, show distance-based parking guidance through three LEDs, drive a barrier-simulation DC motor through timer PWM and a motor driver, control a gate-representation relay, detect button presses using EXTI, manage interrupt priorities through NVIC and transmit distance/status messages through UART.

17. Advantages

Provides simple visual parking guidance; combines two types of sensors; demonstrates timer input capture and PWM; demonstrates GPIO, EXTI, NVIC and UART; can be extended with a display, additional sensors or automatic slot management.

18. Limitations and Safety Notes

Ultrasonic readings can be affected by soft, angled or narrow objects and environmental conditions. IR detection depends on the module and surface conditions. Thresholds are demonstration values and require calibration. The prototype is not a certified automotive safety device.

HC-SR04 ECHO commonly outputs 5 V; protect the STM32 input with a suitable resistor divider or level shifter. Confirm that the divider gives a safe logic level. All circuit grounds must be common.

Do not connect a DC motor or bare relay coil directly to a microcontroller pin. Use a motor driver/relay driver and a suitable external supply. Use flyback protection where required and verify module voltage ratings. Keep motor power separate from the MCU supply where appropriate.

19. Future Enhancements

Add an LCD/OLED display; add buzzer feedback; average multiple distance readings to reduce noise; add limit switches for barrier end positions; implement slot availability counting; add wireless monitoring; add fault detection and safer barrier-stop logic.

20. Conclusion

The Intelligent Parking Assistance System demonstrates how an STM32F103RB microcontroller can combine ultrasonic distance measurement, IR vehicle detection, LED guidance, timer-generated PWM, relay control, external interrupts, NVIC priorities and UART communication. The report provides a proposed wiring and configuration plan for a prototype. Successful completion requires wiring the circuit according to the selected board, implementing the firmware in STM32CubeIDE and recording the actual test results.

21. Final Project Summary

The project uses an STM32F103RB (ARM Cortex-M3) to assist parking in a miniature demonstration setup. An ultrasonic sensor estimates the distance to an obstacle, an IR sensor detects a vehicle, LEDs indicate safe/caution/stop ranges, a timer PWM drives a DC motor through a driver to simulate a barrier, and a relay represents the gate. A push button is handled through EXTI/NVIC, and USART1 sends distance and parking status to a serial terminal.

22. References

• STMicroelectronics, STM32F103x8/xB datasheet.

• STMicroelectronics, RM0008 Reference Manual for STM32F1-series microcontrollers.

• STMicroelectronics, STM32CubeMX and STM32CubeIDE user documentation.

• HC-SR04 ultrasonic ranging module documentation supplied with the module.

• Datasheets for the selected motor driver and relay module.

Appendix A: Example UART Output

Distance: 42 cm | Vehicle: YES | Parking: CAUTION
Distance: 18 cm | Vehicle: YES | Parking: STOP
Vehicle: NO | Parking: IDLE

Appendix B: Implementation Notes

• Distance in centimetres ≈ echo pulse width in microseconds / 58.

• Example bands: GREEN > 100 cm; YELLOW 30–100 cm; RED < 30 cm. Calibrate these values.

• Use timeout handling for missing echoes and a level shifter/divider for a 5 V ECHO signal.

• Use a rated motor driver and relay module; never drive motor/relay coils directly from GPIO.

• Add real test results and circuit photographs only after the prototype is assembled and verified.